

Hamiltonian Neural Networks (HNNs): Theory & PyTorch Implementation

Authors: Ardavan (Ahmad) Borzou & Razie Yousefi

This document is a guide (mini-course) to Hamiltonian Neural Networks (HNNs), a specialized machine learning architecture designed to model systems with high stability and precision.

By learning the system's Hamiltonian—a scalar function representing energy—rather than direct trajectories, HNNs incorporate physical laws as inductive biases.

This guide covers **(i)** the theoretical necessity of HNNs, **(ii)** a practical comparison against conventional Multi-Layer Perceptrons (MLPs) using a pendulum simulation, and **(iii)** a full PyTorch implementation demonstrating how HNNs effectively overcome extrapolation challenges.

Why Do We Need Hamiltonian Neural Networks?

Imagine recording the motion of a simple pendulum and asking a neural network to predict what happens next. A conventional neural network can often predict the next few frames quite accurately. However, if we repeatedly feed its predictions back into itself, small errors accumulate. After enough time, the pendulum may appear to gain energy from nowhere, lose energy mysteriously, or even stop oscillating despite the absence of friction.

This phenomenon is known as **error accumulation** or **trajectory drift**. The problem is that standard neural networks learn patterns in data but are not explicitly taught the laws that generated those patterns. They may fit the training data well while violating important physical constraints.

Hamiltonian Neural Networks were introduced to address this problem. HNNs introduce a strong physical inductive bias by requiring the learned dynamics to arise from a Hamiltonian function. This constraint often improves:

- Long-term stability
- Physical consistency
- Data efficiency
- Extrapolation beyond training trajectories

The original HNN paper demonstrated this on mass-spring systems, pendulums, two-body orbital systems, and even pixel observations of a pendulum.

Real-World Applications

Many important systems evolve according to conservation laws. Examples include:

- Pendulums
- Planetary orbits
- Simple weather systems
- Molecular dynamics
- Robotics
- Some fluids
- Some quantum systems

In all of these domains, accurately predicting motion over long time horizons is crucial.

The Central Idea of HNNs

The central insight behind Hamiltonian Neural Networks is surprisingly simple: **Instead of teaching a neural network to predict how a system moves, teach it to learn the Hamiltonian (total energy function of the system) that generates the motion.**

Once the Hamiltonian is known, the laws of motion can be recovered automatically through differentiation. To understand why this works, we first need a brief introduction to Hamiltonian mechanics, which you will see next.

Phase Space

A physical system is completely described by its state. The concept is similar to that of the latent space in the machine learning realm. For a pendulum, the state consists of:

(q, p)

where:

- q describes the pendulum angle
- p describes the pendulum's angular velocity (time derivative of the angle)

The collection of all possible states forms a geometric space called **phase space**. Every point in phase space represents one possible configuration of the system. As the pendulum evolves, the state moves through phase space and traces a trajectory.

Hamiltonian Function

The Hamiltonian is a scalar-valued function $H(q, p)$ that assigns a numerical value to every point in phase space. For many mechanical systems:

$$H = T + V$$

where:

(T) = kinetic energy

(V) = potential energy

For a simple pendulum:

$$H(q, p) = \frac{p^2}{2ml^2} + mgl(1 - \cos q)$$

where:

(m) = mass

(l) = pendulum length

(g) = gravitational acceleration

Note: In HNNs, we do not need to know how $H(q, p)$ depends on q and p .

Although the Hamiltonian is only a single scalar value, it contains enough information to generate the entire dynamics of the system.

Once the Hamiltonian is known, the laws of motion automatically emerge from its derivatives:

$$dq/dt = \partial H / \partial p$$

$$dp/dt = - \partial H / \partial q$$

Simply speaking, this is how nature operates. So, memorize these equations.

Hamiltonian Neural Net (HNN) VS. Conventional Neural Net (MLP)

HNN can be a simple MLP (or a more complicated architecture). The difference between HNN and a conventional MLP is in the nature of the output layer and how we use that to predict the future of the system.

- **MLP learns:** $(q, p) \rightarrow (dq/dt, dp/dt)$ directly.
- **HNN learns:** $(q, p) \rightarrow H(q, p)$ and obtains $(dq/dt, dp/dt)$ through differentiation.

This is the key innovation: Rather than learning trajectories, the network learns the physical rule that generates trajectories.

Input Data to HNN

Throughout this guide, we use a simple pendulum as our running example. At every instant, the pendulum has an angle and an angular velocity (angle's time derivative). Together, these form the **state** of the system. In physics, the state is the minimum information required to completely determine the future evolution of a system.

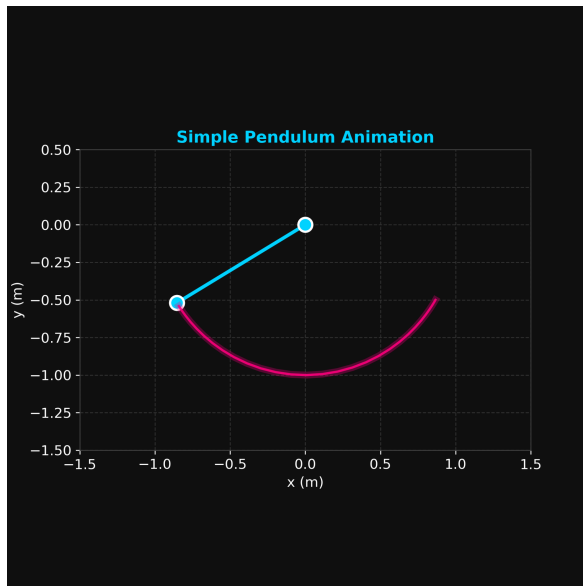


Figure 1: A simple pendulum.

For the simple pendulum, we define the state vector X_i with dimensions 2:

$$X = [q \ p]^T$$

Specifically:

- $X_1 = q$ is the pendulum angle
- $X_2 = p$ is the pendulum angular velocity

More complicated systems may have states (phase spaces) of hundreds, thousands, or even millions of dimensions. For example, a

double pendulum system needs a four-dimensional state consisting of two angles and two angular velocities.

As another example, molecular systems have an extremely high-dimensional phase space. Specifically, we need three variables to describe the position of each molecule and another three variables to describe each molecule's velocity. Therefore, if we have N molecules in our system, the phase space has $6 \times N$ dimensions. If i enumerates each molecule, the Hamiltonian mechanics would be:

$$dq_i/dt = \partial H / \partial p_i$$

$$dp_i/dt = -\partial H / \partial q_i$$

In other words, we have N such equations for each i from 1 to N .

Note: A common approach to solving large-dimensional systems in both physics and machine learning is dimensionality reduction (what physicists call coarse-graining).

Output layer of HNN

Unlike most neural networks, an HNN does not directly output the future state. Instead, it outputs a single scalar quantity:

$$\hat{H}$$

which is the neural network's estimate of the Hamiltonian. We define $\hat{y} = \hat{H}$ with dimension 1. Although its form does not need to be known, for most systems, this scalar is approximately the total energy: potential energy + kinetic energy.

The remarkable idea is that one number (the Hamiltonian function) contains enough information to recover the entire future motion. At first glance, it may seem surprising that the network outputs only a single number. How can one number describe an entire dynamical system?

The answer is that the Hamiltonian is not merely a number. It is a function. The value $H(q, p)$ encodes how energy changes throughout phase space. Once this energy landscape is known, the system's motion can be reconstructed using Hamilton's equations. This dramatically reduces the burden placed on the neural network compared to models that must predict multiple outputs simultaneously.

Input to Output Transformation in HNN

The goal of the neural network is to approximate the unknown Hamiltonian function of the system, i.e., $H_\theta(q, p) \approx H(q, p)$, where H is the true Hamiltonian. The HNN learns a transformation $\phi(X) = \hat{H}$, or equivalently:

$$\hat{H} = H_\theta(q, p)$$

where:

- H_θ is a neural network
- θ are trainable parameters

Note: **(i)** Our dataset does not contain the true values of the Hamiltonian. It contains the true values of time derivatives of (q, p) . **(ii)** The neural network itself never predicts motion directly.

Prediction Steps

Let's, for now, assume that we know the optimal values of the parameters θ of HNN. Here are the steps to predict the future motion of a system, for example, the pendulum, using the HNN:

Step 1: Start with the Current State of the System

Observe, or measure, the current state $X = (q, p)$ of the system and insert it into the input layer of HNN.

Step 2: Estimate Hamiltonian

Pass X through the neural network to get $\hat{H}(q, p)$, which is the total energy assigned to the (q, p) point in the phase space.

Step 3: Compute Hamiltonian Gradients

Use automatic differentiation (backpropagation) to compute:

- $\partial \hat{H} / \partial q$ (how energy changes with angle)
- $\partial \hat{H} / \partial p$ (how energy changes with angular velocity)

Step 4: Apply Hamilton's Equations

Find the time derivatives of (q, p) using equations of motion (laws of nature) described above:

$$dq/dt = \partial \hat{H} / \partial p$$

$$dp/dt = - \partial \hat{H} / \partial q$$

Step 5: Obtain State Derivatives

Place the results into a vector $S_H = (dq/dt, dp/dt)$. This vector describes the instantaneous direction of motion in phase space.

Step 6: Integrate Through Time

The derivatives are integrated to produce full trajectories:

$$(q_{t+\Delta t}, p_{t+\Delta t}) = (q_t, p_t) + \int S_H dt$$

Numerical methods such as Euler, Runge-Kutta (RK4), or the state-of-the-art symplectic Integrators are typically used to compute the integral.

Training Process of HNNs

In the previous section, we assumed the optimal values of the network parameters θ were already known. In reality, we always start with an initial guess for the parameters and use data to find the optimal values through methods like gradient descent.

The dataset contains measured (true values of) (q, p) pairs and their derivatives $(dq/dt, dp/dt)$. The true derivatives may originate from:

- simulations
- direct experiments and observations
- numerical differentiation of observed trajectories

Note: The dataset does not contain the true Hamiltonian.

Loss Function

The objective is to minimize HNN loss, which is defined as the difference between predicted (by the Hamiltonian equations discussed above) and true time derivatives:

$$L_{HNN} = |\partial H_\theta / \partial p - dq/dt|^2 + |-\partial H_\theta / \partial q - dp/dt|^2$$

In other words, the network is judged by the quality of the physics (dynamics) it

produces, rather than comparing Hamiltonians directly.

Original Article's Implementation Settings:

Here are the settings used to train some of the systems presented in the original arXiv article:

- **Learning Rate:** 10^{-3}
- **Optimizer:** Adam
- **Architecture:** Fully-connected networks (MLP)
- **Activation:** Tanh

Computational Complexities VS. Advantages

The computational cost is slightly higher than a standard network due to the additional automatic differentiation step required to compute $\partial H/\partial q$ and $\partial H/\partial p$. Memory requirements are also larger because the computation graph must be retained.

However, the model generalizes with much less data, which substantially reduces the computations. Taking all these into account, HNNs are the winners in computational cost efficiency.

Not to mention that in this comparison, we assumed high-quality data is available to train conventional neural nets. This is not always true. For example, in a planetary system, we need to wait decades or centuries to collect more data. In some robotic systems, observing a part of the trajectory might be much cheaper than observing another part.

Limitations

HNNs assume a **conserved quantity** exists. In real systems with friction or air resistance, energy is not conserved, and a pure HNN may incorrectly force conservation. In these scenarios, a more appropriate dynamics is needed. For example, we might be able to use the Fokker-Planck equation instead of Hamiltonian dynamics.

Extensions and Variants

This field of research has already gone a long way. There are many publications on the following topics:

- **Lagrangian Neural Networks (LNNs):** Learn the Lagrangian instead of the Hamiltonian.
- **Symplectic Neural Networks:** Focus on preserving geometric structure.
- **Dissipative HNNs:** Handle systems with friction and external forces.

Comparison of HNN and MLP in Practice (PyTorch)

Overview

In the following, we simulate a simple pendulum to generate a dataset of its states (q, p) , which are the angles and angular velocities, and their time derivatives $(dq/dt, dp/dt)$.

To test the generalization capabilities of neural networks, we created an extreme extrapolation scenario: we partitioned the dataset and restricted training strictly to states where the angle is negative ($q < 0$). We then trained two different architectures:

1. **Baseline MLP (Conventional Neural Network):** A standard feedforward network that takes (q, p) as input and directly predicts the state derivatives $(dq/dt, dp/dt)$.
2. **Hamiltonian Neural Network (HNN):** A physics-based architecture that outputs a scalar value representing the system's Hamiltonian $H(q, p)$, and computes the state derivatives using Hamilton's equations via PyTorch's automatic differentiation.

Results

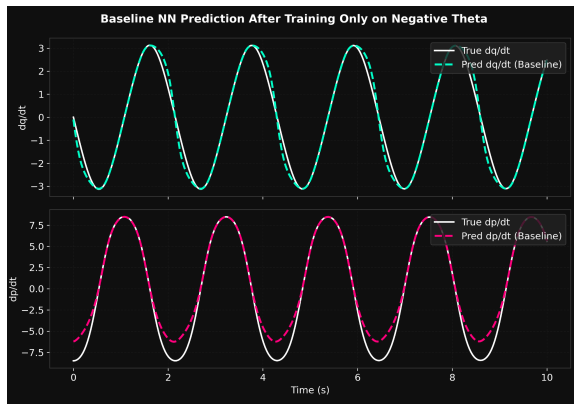


Figure 2: MLP prediction for the simple pendulum.

As seen in Figure 2, the standard MLP performs well within its training domain but fails to generalize. When the pendulum swings into the unseen positive angle region ($q \geq 0$), the network's predictions for dp/dt diverge from the true physics. Since standard

MLPs are purely data-driven function approximators, they merely fit the statistical distribution of the training data. **Lacking any domain knowledge of the system, conservation of energy, or symmetry, the MLP is unable to accurately extrapolate outside the $q < 0$ subset.**

The Hamiltonian Neural Network successfully overcomes this limitation by incorporating physics directly into its learning process. In addition to the Hamiltonian dynamics, we embedded an explicit physical symmetry into our forward pass:

$$H(x) = 1/2 (H(x) + H(-x)).$$

Because the physics of a simple pendulum are symmetric, learning the left side implicitly reveals the right side.



Figure 3: HNN prediction for the simple pendulum.

As demonstrated in Figure 3, these physical constraints force the model to capture the true underlying dynamics of the system. Rather than memorizing local data points, it discovers the conserved Hamiltonian rules, allowing it to perfectly predict the

pendulum's motion even in the entirely unseen positive region.

Datasets

To generate the dataset, we first used a mathematical solver to simulate the motion of a simple pendulum as it swung for 10 seconds, starting from an initial angle of 60 degrees.

For every frame of this simulation, we captured the pendulum's exact state, specifically its angle and angular velocity.

We then applied real physics formulas to calculate the precise rate of change for both the angle and angular velocity at each moment, establishing the derivatives that the neural network would learn to predict.

Finally, to rigorously test the neural nets' ability to learn and generalize, we deliberately unfairly partitioned the dataset. We trained the neural network exclusively on data from the left side of the pendulum's arc, where the angles are negative, while completely hiding all data from the right side (positive angles) to observe whether the model could deduce the underlying physics and accurately predict motion in that unseen region.

The following Python code generates the datasets:

Python

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.integrate import solve_ivp

import torch

import torch.nn as nn

import torch.optim as optim

# Physical constants
```

```
g = 9.81 # Gravity (m/s^2)
L = 1.0 # Length of the pendulum (m)

# Equation of motion:  $\theta'' = -(g/L) * \sin(\theta)$ 
# Let  $y = [\theta, \omega]$ , then  $y' = [\omega, -(g/L) * \sin(\theta)]$ 
def pendulum_dynamics(t, y):
    theta, omega = y
    dydt = [omega, -(g/L) * np.sin(theta)]
    return dydt

# Initial conditions
theta0 = np.pi / 3 # Initial angle (60 degrees)
omega0 = 0.0 # Initial angular velocity (rad/s)
y0 = [theta0, omega0]

# Time array
t_span = (0, 10) # Simulate for 10 seconds
fps = 30 ## frames per second
frames = int((t_span[1] - t_span[0]) * fps)
t_eval = np.linspace(t_span[0], t_span[1], frames)
```

```
# Solve the ODE

sol = solve_ivp(pendulum_dynamics, t_span, y0, t_eval=t_eval,
method='RK45')

# Prepare full dataset from the simulation

# q = theta, p = omega (generalized coordinates)

q_all = torch.tensor(sol.y[0], dtype=torch.float32).unsqueeze(1)
p_all = torch.tensor(sol.y[1], dtype=torch.float32).unsqueeze(1)

# True time derivatives from our physics equations

dq_dt_all = p_all
dp_dt_all = -(g/L) * torch.sin(q_all)

# Use only the left side (negative theta) for training.

# This intentionally creates an extrapolation task for theta >= 0.

neg_mask = (q_all[:, 0] < 0)

q_train = q_all[neg_mask]
p_train = p_all[neg_mask]
dq_dt_train = dq_dt_all[neg_mask]
```

```
dp_dt_train = dp_dt_all[neg_mask]

# Inputs and targets for PyTorch
X_train = torch.cat([q_train, p_train], dim=1) # Shape: [N, 2]
Y_train = torch.cat([dq_dt_train, dp_dt_train], dim=1) # Shape: [N, 2]

X_all = torch.cat([q_all, p_all], dim=1)
Y_all = torch.cat([dq_dt_all, dp_dt_all], dim=1)
```

PyTorch Train & Test: Baseline MLP Model

The following Python code trains and tests the baseline MLP:

Python

```
class BaselineNN(nn.Module):

    def __init__(self, hidden_dim=64):

        super(BaselineNN, self).__init__()

        self.net = nn.Sequential(

            nn.Linear(2, hidden_dim),

            nn.Tanh(),

            nn.Linear(hidden_dim, hidden_dim),
```

```
        nn.Tanh(),
        nn.Linear(hidden_dim, 2) # Outputs dq/dt and dp/dt
    directly
)

def forward(self, x):
    return self.net(x)

# Initialize baseline model and optimizer
baseline_model = BaselineNN()

baseline_optimizer = optim.Adam(baseline_model.parameters(),
                                lr=1e-3)

# Training loop
baseline_epochs = 1200
baseline_losses = []

print("Training Conventional Neural Network...")
for epoch in range(baseline_epochs):
    baseline_optimizer.zero_grad()
    y_pred_train = baseline_model(X_train)
```



```
loss = nn.MSELoss()(y_pred_train, Y_train)

loss.backward()

baseline_optimizer.step()


baseline_losses.append(loss.item())


# Predict vector field on the full trajectory (including unseen
theta >= 0)

y_pred_all_baseline = baseline_model(X_all).detach().numpy()
y_true_all_np = Y_all.detach().numpy()
theta_all_np = q_all.detach().numpy().squeeze()


pos_mask_np = theta_all_np >= 0
neg_mask_np = theta_all_np < 0


mse_all_baseline = np.mean((y_pred_all_baseline - y_true_all_np) **
2)

mse_neg_baseline = np.mean((y_pred_all_baseline[neg_mask_np] -
y_true_all_np[neg_mask_np]) ** 2)

mse_pos_baseline = np.mean((y_pred_all_baseline[pos_mask_np] -
y_true_all_np[pos_mask_np]) ** 2)
```

PyTorch Train & Test: HNN Model

The following Python code trains and tests the HNN model:

Python

```
class HNN(nn.Module):

    def __init__(self, hidden_dim=64):

        super(HNN, self).__init__()

        self.net = nn.Sequential(

            nn.Linear(2, hidden_dim),

            nn.Tanh(),

            nn.Linear(hidden_dim, hidden_dim),

            nn.Tanh(),

            nn.Linear(hidden_dim, 1) # Outputs a scalar Hamiltonian
(energy)

        )

    def forward(self, x):

        # Enforce  $H(q, p) = H(-q, -p)$ .

        H_x = self.net(x)

        H_neg_x = self.net(-x)

        return 0.5 * (H_x + H_neg_x)
```

```
def hnn_loss(model, x, y_true):  
    x = x.clone().requires_grad_(True)  
    H = model(x)  
  
    # Compute gradients of H with respect to inputs [q, p]  
    dH = torch.autograd.grad(H.sum(), x, create_graph=True)[0]  
  
    # Hamilton's equations: dq/dt = dH/dp, dp/dt = -dH/dq  
    # dH[:, 0] is gradient wrt q, dH[:, 1] is gradient wrt p  
    dq_dt_pred = dH[:, 1]  
    dp_dt_pred = -dH[:, 0]  
  
    y_pred = torch.stack([dq_dt_pred, dp_dt_pred], dim=1)  
  
    # Train to match the true time derivatives  
    loss = nn.MSELoss()(y_pred, y_true)  
    return loss  
  
def hnn_vector_field(model, x):
```

```

x_req = x.clone().requires_grad_(True)

H = model(x_req)

dH = torch.autograd.grad(H.sum(), x_req, create_graph=False)[0]

dq_dt_pred = dH[:, 1]
dp_dt_pred = -dH[:, 0]

return torch.stack([dq_dt_pred, dp_dt_pred], dim=1)

# Initialize model and optimizer

model = HNN()

optimizer = optim.Adam(model.parameters(), lr=1e-3)

# Training loop

epochs = 1200

losses = []

print("Training Hamiltonian Neural Network...")

print(f"Training samples (theta < 0): {X_train.shape[0]} / {X_all.shape[0]}")

for epoch in range(epochs):

```

```
optimizer.zero_grad()

loss = hnn_loss(model, X_train, Y_train)

loss.backward()

optimizer.step()

losses.append(loss.item())

if (epoch + 1) % 200 == 0:

    print(f"Epoch {epoch + 1}/{epochs}, Loss:
{loss.item():.6f}")

# Predict vector field on the full trajectory (including unseen
theta >= 0)

y_pred_all = hnn_vector_field(model, X_all).detach().numpy()
y_true_all = Y_all.detach().numpy()
theta_all = q_all.detach().numpy().squeeze()

pos_mask_np = theta_all >= 0
neg_mask_np = theta_all < 0

mse_all = np.mean((y_pred_all - y_true_all) ** 2)
```

```
mse_neg = np.mean((y_pred_all[neg_mask_np] -
y_true_all[neg_mask_np]) ** 2)

mse_pos = np.mean((y_pred_all[pos_mask_np] -
y_true_all[pos_mask_np]) ** 2)

dp_pred_pos_region = y_pred_all[pos_mask_np, 1]
dp_true_pos_region = y_true_all[pos_mask_np, 1]

# If this predicted rate is near 0 while true is higher, the model
is missing

# negative dp/dt behavior on the unseen (theta >= 0) side.

pred_negative_rate = np.mean(dp_pred_pos_region < 0)
true_negative_rate = np.mean(dp_true_pos_region < 0)

print(f"MSE on all states: {mse_all:.6e}")
print(f"MSE on trained region (<0): {mse_neg:.6e}")
print(f"MSE on predicted region (>=0): {mse_pos:.6e}")
print(f"dp/dt<0 rate in theta>=0 region | true:
{true_negative_rate:.3f}, pred: {pred_negative_rate:.3f}")
```